

PrithviNet Hackathon Execution Plan

Date: 13 March 2026

Goal

Complete a stable, demoable PrithviNet build for the hackathon with one strong end-to-end story:

- pollution data ingestion
- live dashboard updates
- threshold and anomaly alerts
- short-term forecasting
- public and regulator views

Recommended Scope

Focus on air pollution as the polished primary flow.

Treat water and noise as supported extensions through the same backend model instead of trying to make all three equally deep in 36 hours.

36-Hour Plan

Phase 1: First 2 Hours

1. Start infrastructure with Docker for Postgres and Redis.
2. Set up backend environment variables in ``backend/.env``.
3. Install frontend and backend dependencies.
4. Seed the database with sample industries, locations, limits, and readings.
5. Confirm the backend health endpoint and frontend app both run.

Success condition:

- backend responds on ``/health``
- frontend loads without crashing
- seeded data exists

Phase 2: Hours 3 to 8

1. Verify the readings ingestion path works.
2. Verify the forecast endpoint returns data for seeded locations.
3. Verify the public overview dashboard shows map and AQI-style summary.
4. Verify the regulator or admin dashboard renders the key compliance and alerts views.
5. Remove or hide broken pages instead of fixing low-value features.

Success condition:

- one dashboard path works cleanly
- one forecast path works cleanly
- one alert path works cleanly

Phase 3: Hours 9 to 18

1. Import the real air pollution dataset.
2. Normalize the dataset into the project schema: location, parameter, value, timestamp, source.
3. Add any missing monitoring units and limits required by the imported parameters.
4. Validate charts and public views with the real imported data.
5. Keep Prophet forecasting for the air time series.

Success condition:

- real air data is visible in the platform
- forecast runs on usable historical series

Phase 4: Hours 19 to 26

1. Import noise data only if it is already clean and timestamped.
2. Add noise monitoring locations and parameter definitions.
3. Use rule-based thresholds and simple anomaly flags for noise.
4. Import water data only if it is equally clean and does not break the schedule.

Success condition:

- air flow is polished
- noise or water is shown as an additional supported stream only if integration is low-risk

Phase 5: Hours 27 to 32

1. Polish the demo flow.
2. Make the public page visually stable.
3. Make the forecast chart understandable.
4. Make alert states obvious.
5. Prepare one scripted demo path from live data to alert to forecast.

Success condition:

- no obvious crashes
- charts are readable
- demo path is predictable

Phase 6: Final 4 Hours

1. Freeze features.
2. Fix only bugs and broken data mappings.
3. Update README and submission notes.
4. Prepare screenshots.
5. Rehearse a 2-minute demo and 30-second technical explanation.

Success condition:

- stable demo
- clear pitch
- no last-minute model experiments

What Not To Do

- do not train a custom deep model from scratch
- do not try to productionize auth
- do not redesign every page
- do not add multiple new services late in the hackathon
- do not spend hours on perfect infrastructure

ML Recommendation

Use what already exists in the project:

- Prophet for short-term forecasting
- threshold-based compliance alerts
- lightweight anomaly logic for suspicious readings

This is the right tradeoff for a 36-hour hackathon. A finished, believable analytics pipeline is stronger than an unfinished custom-trained model.

Datasets To Send

You should send datasets in CSV format first. That is the fastest format to map into the current backend.

Air Pollution Dataset

Minimum required columns:

- `timestamp`
- `location\_name` or `sensor\_id`
- `latitude`

- `longitude`
- `parameter`
- `value`
- `unit`

Strongly preferred columns:

- `region`
- `industry\_name`
- `pollution\_type`
- `source`
- `quality\_flag`

Best parameters to include:

- `PM2.5`
- `PM10`
- `NO2`
- `SO2`
- `CO`
- `O3`
- `NH3`

For this project, the best air dataset has:

- hourly or 15-minute timestamps
- at least 30 days of history per location
- consistent units
- fewer missing rows

Noise Pollution Dataset

Minimum required columns:

- `timestamp`
- `location\_name` or `sensor\_id`
- `latitude`
- `longitude`
- `value`
- `unit`

Preferred columns:

- `noise\_metric`

- `zone\_type`
- `source`
- `quality\_flag`
- `region`

Best noise metrics to include:

- `Leq`
- `Lmax`
- `Lmin`
- `day\_avg`
- `night\_avg`

For this project, noise data is useful when it has:

- regular timestamps
- clear unit such as `dB` or `dB(A)`
- zone or area context such as industrial, residential, silence, or traffic

Water Pollution Dataset

Minimum required columns:

- `timestamp`
- `location\_name` or `sensor\_id`
- `latitude`
- `longitude`
- `parameter`
- `value`
- `unit`

Good parameters to include:

- `pH`
- `DO`
- `BOD`
- `COD`
- `TDS`
- `turbidity`
- `nitrate`
- `lead`

Water data is worth integrating only if it is already clean, because it usually needs more parameter mapping and threshold setup than air.

Best Dataset Shape For This Repo

The cleanest import shape is one row per observation:

```
timestamp, location_name, latitude, longitude, parameter, value, unit, source
```

Example:

```
2026-03-01T10:00:00Z, Central Station, 21.2514, 81.6296, PM2.5, 84.2, ug/m3, sensor
```

Avoid wide spreadsheets like this if possible:

```
timestamp, pm25, pm10, no2, so2, ...
```

Long format is much easier for this backend.

What To Send Me Next

Send these for each dataset:

1. one sample CSV file
2. number of rows
3. date range covered
4. list of columns
5. unit definitions
6. whether timestamps are hourly, daily, or irregular
7. whether each row has coordinates or only location names

If you send that, I can tell you very quickly:

- which dataset should be integrated first
- whether forecasting is possible
- whether anomaly detection is possible
- what fields must be cleaned before import

Recommended Final Demo Story

1. Show live or seeded air pollution readings on the map.
2. Show a location trend line.
3. Show a 72-hour forecast.
4. Trigger or display an alert for a high reading.
5. Show the regulator dashboard with compliance context.

That is the highest-value hackathon version of this project.